تمرين: جدول الضرب والعد التنازلي والعدّاد والمجموع

الدرس 15 — SST-ICT-01، 1IOT — أربعة برامج مع التحقق من العدد وحلقة for

02/11/2026

1 ماذا فعلنا اليوم

لا يوجد اليوم موضوع جديد: كان درس **تمارين**، كلها مكتوبة على **onlinegdb**. أنجزنا **أربعة برامج**، اثنان معًا، واثنان أولًا كل واحد بمفرده ثم صححناهما معًا:

1. **جدول الضرب** (**tabellina**) لعدد ما (الفقرة 2)؛
2. **العد التنازلي** (**conto alla rovescia**) (الفقرة 3)؛
3. **كم مرة تدور** حلقة من البداية إلى النهاية: **العدّاد** (**contatore**) (الفقرة 5)؛
4. **مضاعفات** عدد حتى 100 و**مجموعها** (**somma**) (الفقرة 6).

في الوسط تحدثنا عن **الذكاء الاصطناعي** (الفقرة 4)، وفي النهاية عن **طريقة الدراسة** (الفقرة 8).

للبرامج الأربعة **الشكل نفسه**: أولًا **أطلب عددًا** وأتحقق من صحته، ثم أستخدم **حلقة for**، ثم **أطبع النتيجة**. إذا فهمت هذا الشكل، فتمارين الواجب المنزلي هي الشيء نفسه بأعداد مختلفة.

2 البرنامج الأول: جدول الضرب

مثال

المطلوب (الواجب C02، التمرين 1). اكتب برنامجًا يطلب عددًا من 1 إلى 10 (إذا كان خاطئًا يتكرر السؤال) ويطبع على سطر واحد جدول ضربه: من n حتى 10 مرات n.

2.1 التحقق من العدد: do while

أحتاج إلى متغير `int n`. يجب طرح السؤال **مرة واحدة على الأقل** و**تكراره** إذا كان العدد خاطئًا: لذلك أستخدم **do while**.

```c
do
{
    printf("Inserisci un numero da 1 a 10: ");
    scanf("%d", &n);
} while ((n < 1) || (n > 10));
```

تنبيه

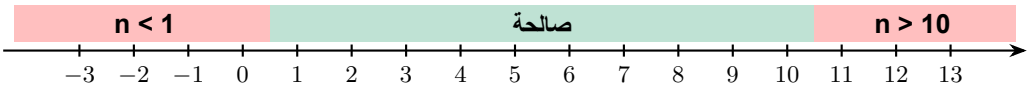
كثيرًا ما يخطئ الطلاب في scanf. عند تصحيح الواجبات يتكرر خطآن باستمرار:

- تنقص **الفاصلة** بعد علامتي التنصيص: `scanf("%d" &n)` خطأ؛
- تُغلق **علامتا التنصيص** مباشرة بعد d، وليس في النهاية: `scanf("%d, &n")` خطأ.

الصيغة الصحيحة واحدة فقط: `scanf("%d", &n);`

2.2 متى يكون خاطئًا؟ الرسم على المستقيم

عندما أكتب الشرط يساعدني **رسم**. أرسم **مستقيمًا** وأضع عليه الأعداد: 0، وعلى اليمين 1 و2 و3 وهكذا، وعلى اليسار -1 و-2 و-3. ثم ألوّن منطقة الأعداد **الصالحة** (من 1 إلى 10) ومناطق الأعداد **الخاطئة**.



شكل 1: مستقيم الأعداد: يتكرر السؤال عندما يقع العدد في إحدى المنطقتين الحمراوين.

يتكرر السؤال عندما أكون في المناطق **الحمراء**: عندما يكون n **أصغر من 1 أو** n **أكبر من 10**. تُكتب "أو" هكذا `||`، وهذا هو الشرط: `(n < 1) || (n > 10)`. الأقواس حول الجزأين **ليست إجبارية**: توضع لزيادة الوضوح.

- **انتبه إلى التفكير**: في do while أكتب متى **أكرر**، أي عندما يكون العدد **خاطئًا**. لو كتبت `(n >= 1) && (n <= 10)`، الذي يصف المنطقة الخضراء، لكرر البرنامج السؤال عندما يكون العدد صحيحًا بالذات. بـ `&&` ("و") توصف منطقة **محصورة** بين قيمتين؛ وبـ `||` ("أو") المنطقتان **الخارجيتان**.
- يصلح أيضًا `n >= 0` بدل `n < 1`: مع الأعداد **الصحيحة** هما الشيء نفسه. لكن ليس مع الأعداد العشرية: 0,99 أصغر من 1 لكنه ليس أصغر من 0 أو يساويه.
- ستجدون طريقة رسم المناطق هذه على المستقيم في الرياضيات مع **المتراجحات** (**disequazioni**).

2.3 قبل كتابة for: مثال بالأعداد

بعد do while يوجد سطر `printf("\n");` هو **غير ضروري**: عند الضغط على Invio ينتقل المؤشر أصلًا إلى السطر التالي. يضيف فقط سطرًا فارغًا لأجل التنسيق. ليس خطأ ونتركه.

قبل كتابة الحلقة نأخذ مثالًا لنفهم **ما هي القاعدة**. إذا كانت n تساوي 2، أريد رؤية هذه الأعداد:

العدد المطبوع	2	4	6	8	10	12	14	16	18	20
كيف نحصل عليه	$2 \cdot 1$	$2 \cdot 2$	$2 \cdot 3$	$2 \cdot 4$	$2 \cdot 5$	$2 \cdot 6$	$2 \cdot 7$	$2 \cdot 8$	$2 \cdot 9$	$2 \cdot 10$
قيمة i	1	2	3	4	5	6	7	8	9	10

العدد الذي يتغير هو الثاني: نحتاج إلى **متغير جديد** `i`، يبدأ من **1** ويصل إلى **10** (مع 10). العدد المطبوع هو دائمًا `i * n`.

2.4 إيجاد الخطأ بالتفكير

لنجرب كتابة حد الحلقة هكذا: إذا كانت n تساوي 2 أريد الوصول إلى 20، وإذا كانت 7 إلى 70، إذن `i <= n * 10`. يبدو صحيحًا. لنجرب.

```
for (int i = 1; i <= n * 10; i++)
{
    printf("%d ", n * i);
}
```

```
Inserisci un numero da 1 a 10: 15
Inserisci un numero da 1 a 10: 12
Inserisci un numero da 1 a 10: -6
Inserisci un numero da 1 a 10: 2

2 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40
```

التحقق من العدد **يعمل**: رُفضت 15 و12 و-6. لكن الأعداد وصلت إلى **40** بدل 20. أين الخطأ؟ نفكر في المعطيات:

- مع n = 2 يطبع **20** عددًا بدل 10، أي الضعف؛
- مع n = 3 يطبع 30، أي ثلاثة أضعاف، ويصل إلى 90؛
- مع n = 10 يطبع 100، ويصل إلى 1000.

يطبع أعدادًا أكثر من اللازم **n مرة**: المذنب هو `10 * n`. لنعد إلى الجدول: `i` تذهب **من 1 إلى 10** مهما كانت قيمة n. الحد هو ببساطة **10**. لا يعتمد على n: النتيجة `i * n` هي التي تعتمد على n.

لهذا لا تصلح التصحيحات الأخرى التي تخطر بالبال:

- `i <= 5` تصلح للعدد 2 فقط: هي نصف 10 فقط لأننا ننظر إلى 2، ومع 3 ستطبع حتى 15 فقط؛
- `i <= i * 10` لا تنتهي أبدًا: `i` تتغير في كل دورة، والحد يتحرك معها؛
- `i += 2` ("لأنها أعداد زوجية") غير لازمة: الأعداد الزوجية تأتي من الضرب `i * n`، أما `i` فيجب أن تعد 1، 2، 3.

ملاحظة

البرنامج الخاطئ **ليس مأساة**. إذا وجدت خطأ **أعود إلى خطواتي** وأفهم **أين** أخطأت في التفكير. كتابة جدول بالأعداد التي أتوقعها تساعد كثيرًا: هنا يظهر فورًا أن `i` تذهب دائمًا من 1 إلى 10. لا يكفي قول "خطأ": يجب أن نفهم **ما الذي** لا يعمل، وأن نجربه.

2.5 البرنامج الكامل

```c
#include <stdio.h>

int main()
{
    int n;

    do
    {
        printf("Inserisci un numero da 1 a 10: ");
        scanf("%d", &n);
    } while ((n < 1) || (n > 10));

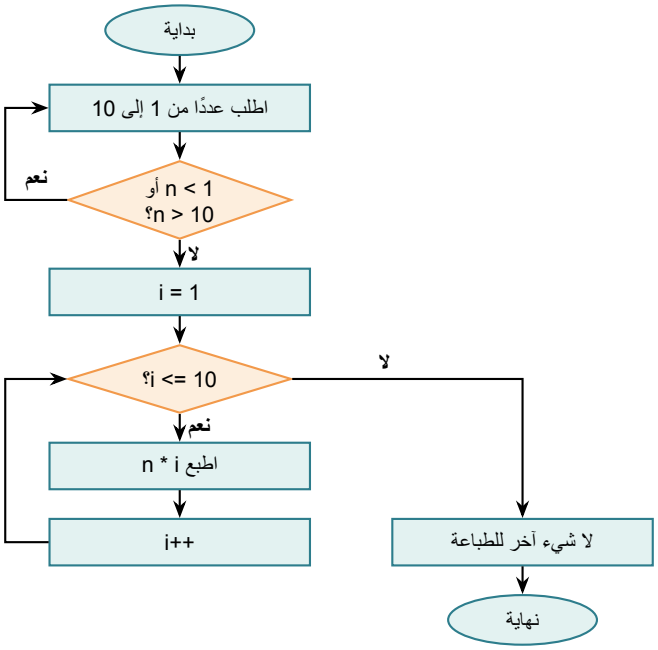
    printf("\n");

    for (int i = 1; i <= 10; i++)
    {
        printf("%d ", n * i);
    }

    return 0;
}
```

```
Inserisci un numero da 1 a 10: 2

2 4 6 8 10 12 14 16 18 20
```



شكل 2: مخطط انسيابي لجدول الضرب.

2.6 ماذا يفعل printf فعلًا

printf **ليس مجرد إجراء شكلي**: هو الذي **يعرض على الشاشة** النتائج. إذا حذفت `printf` من الحلقة، يدور البرنامج الدورات نفسها لكن **لا يظهر شيء**.

يقرر printf أيضًا **كيف** تظهر البيانات، أي **ينسقها** (**formatta**). ما أكتبه بين علامتي التنصيص يظهر على الشاشة: إذا كتبت داخلها ciao، تظهر ciao في كل دورة.

- `printf("%d", n * i);` يلصق الأعداد كلها متتالية: 2468101214…؛
- `printf("%d ", n * i);` يترك **مسافة** بعد كل عدد: 2 4 6 8 …؛
- `printf("%d\n", n * i);` ينتقل إلى **سطر جديد** بعد كل عدد: يضعها في **عمود**.

وإذا أردنا رؤية الحساب أيضًا، 2 * 1 = 2؟ يمكن وضع **عدة قيم** في printf واحد: كل %d يمثل **العدد الموجود في متغير** أو نتيجة حساب، والقيم بعد الفاصلة تأخذ مكان %d **بالترتيب** الذي كُتبت به. أول %d هو n، والثاني i، والثالث n * i.

```
for (int i = 1; i <= 10; i++)
{
    printf("%d * %d = %d\n", n, i, n * i);
}
```

```
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
...
2 * 10 = 20
```

يمكن الحصول على النتيجة نفسها بـ **ثلاثة printf**، مع وضع \n في الأخير فقط:

```
for (int i = 1; i <= 10; i++)
{
    printf("%d * ", n);
    printf("%d = ", i);
    printf("%d\n", n * i);
}
```

انتبه: هذا **شكل بديل** لفهم printf. طلب التمرين "**على سطر واحد**"، لذلك النسخة التي تُسلَّم هي البرنامج الكامل أعلاه.

3 البرنامج الثاني: العد التنازلي

مثال

المطلوب. اطلب من المستخدم إدخال عدد من 1 إلى 20 (تحقق من صحة الإدخال). اطبع كل الأعداد من العدد المدخل حتى 1 على السطر نفسه. في النهاية اطبع على سطر جديد "Via!".

هذا قمتم به **أولًا بمفردكم**. يشبه كثيرًا البرنامج السابق: do while هو نفسه، تتغير الأعداد فقط. أما for فجديد، لأنه يجب أن **ينزل**. لنأخذ مثالًا مع n = 12: يجب أن أطبع 12، 11، 10، … حتى 1، ثم على سطر جديد Via!.

نستخرج أجزاء for الثلاثة هكذا:

- **من أين يبدأ**: من العدد المدخل. 12 مجرد مثال والعدد موجود في n، إذن int i = n، وليس int i = 12؛
- **إلى متى يستمر**: ما دام i >= 1. يجب طباعة 1 أيضًا، إذن "أكبر من أو يساوي"؛
- **كيف يتغير**: كما يرفع i++ بمقدار 1، كذلك i-- (**decremento**) ينزل بمقدار 1.

تنبيه

شرط for ليس النقطة التي نتوقف عندها: إنه شرط الاستمرار. عندما تصعد الحلقة نكتب عادة `n >= i`. وعندما تنزل **تنقلب** الإشارة: `1 =< i`. لو كتبت `1 =< i`، مع 12 يكون الشرط خاطئًا فورًا و**لا تبدأ** الحلقة أصلًا. ولو كتبت `1 > i`، لن يُطبع 1.

```c
#include <stdio.h>

int main()
{
    int n;

    do
    {
        printf("Inserisci un numero da 1 a 20: ");
        scanf("%d", &n);
    } while ((n < 1) || (n > 20));

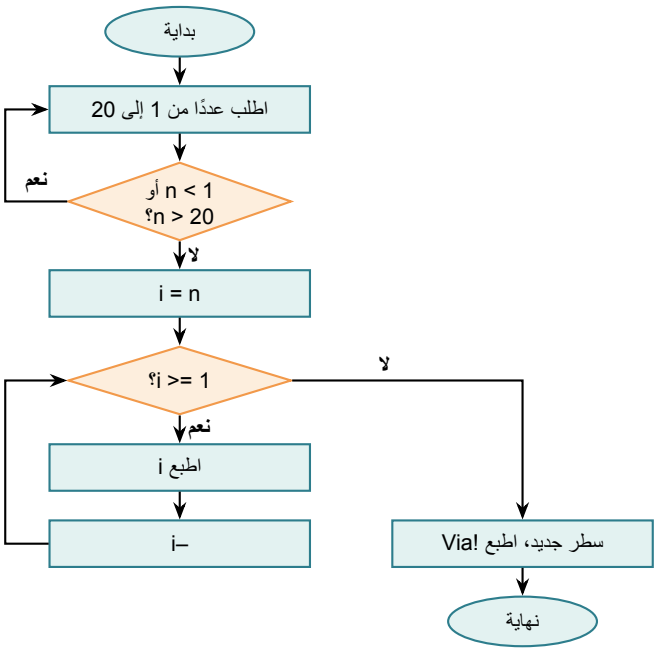
    for (int i = n; i >= 1; i--)
    {
        printf("%d ", i);
    }

    printf("\nVia!");

    return 0;
}
```

```
Inserisci un numero da 1 a 20: 12
12 11 10 9 8 7 6 5 4 3 2 1
Via!
```

الرمز `\n` قبل `Via!` ليس تفصيلًا: **كان المطلوب "سطر جديد"**. بدونه تلتصق `Via!` بالعدد 1. يكون printf الخاص بـ `Via!` **خارج الحلقة**، لأنه يجب أن يظهر مرة واحدة. **اقرؤوا المطلوب جيدًا**: ما لا يُنجز كما طُلب يُحسب خطأ.



شكل 3: مخطط انسيابي للعد التنازلي.

4 الذكاء الاصطناعي: لماذا ليس لحل الواجبات

ما تسلّمونه **يُفحص**، أيضًا لمعرفة هل كُتب بالذكاء الاصطناعي، وهذا يُكتشف. لكن هذه ليست المسألة. لا أقول إنه **لا يجب استخدامه**: هذا غير واقعي، فهو الآن في كل مكان، وهذا طبيعي. المشكلة أنه **يجب تعلّم استخدامه**.

ChatGPT وClaude وPerplexity وغيرها **نماذج رياضية**: برامج تعمل **بالإحصاء**، أي **بالاحتمالات**.

مثال

حجر النرد. أرمي حجر نرد: احتمال ظهور 4 هو

$$\frac{1}{6} \approx 16{,}7\%$$

إذا رميته مرتين، فاحتمال ظهور 4 **في المرتين أقل**، وليس أكبر:

$$\frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36} \approx 2{,}8\%$$

الأكبر هو فقط احتمال ظهوره **مرة واحدة على الأقل** في رميتين:

$$1 - \frac{5}{6} \cdot \frac{5}{6} = \frac{11}{36} \approx 30{,}6\%$$

يقوم مساعد الذكاء الاصطناعي **بقراءة** ما تكتبه و**يحسب** الإجابة الأكثر احتمالًا. لكن معرفة أن 4 يظهر مرة من ست مرات **لا تضمن** ظهوره الآن. وبالمثل، كون الإجابة الأكثر احتمالًا **لا يضمن أنها صحيحة**.

- ما دمت تلعب، فلا مشكلة. إذا اتخذت **قرارات مهمة** بناءً على ما يقوله، فضع في حسابك أن **خطأ سيأتي** عاجلًا أو آجلًا.
- إذا كان ثمن الخطأ يُدفع، فأنت **من يدفعه**: الآلة لا مسؤولية عليها. "قال لي الذكاء الاصطناعي" ليس عذرًا.
- أنا أستخدمه كثيرًا، لكنني **أعرف مسبقًا** هذه الأمور ودرستها: يفيدني في **تسريع** العمل، وأفهم متى يخطئ.
- أنتم هنا لتتعلموها: تنقصكم بعد **المعرفة** لتفهموا هل ما يقوله صحيح أم خاطئ. إذا كتب هو البرنامج **فلن تتعلموا**. أولًا نتعلم، ثم نستخدمه كاملًا.

ملاحظة

سؤال الذكاء الاصطناعي **ليس ممنوعًا**، بشرط أن نفهم بعد ذلك **السبب والكيفية**. يعرف المساعد أكثر منا بكثير وغالبًا ما يقترح حلولًا **لم نرها في القسم**: إذا ظهر شيء كهذا في واجب، فواضح أنه ليس منك. إذا علقت، أرسل له الكود الخاص بك واكتب: **"non darmi il codice, spiegami"** (لا تعطني الكود، اشرح لي). ثم قم بالتعديل بنفسك. عندما تذهبون إلى العمل ستستخدمونه: لكن يجب أن تعرفوا كيف تصححونه حين يقول شيئًا خاطئًا.

5 البرنامج الثالث: كم مرة تتكرر؟ العدّاد

مثال

المطلوب (الواجب C02، التمرين 2). اطلب من المستخدم عدد البداية وعدد النهاية والخطوة (**passo**) (أكبر من 0، مع التحقق). اطبع الأعداد من البداية حتى ما قبل النهاية، وفي الأخير كم مرة تكررت.

5.1 ما هي الخطوة

الخطوة هي **بكم** **أتقدم** في كل مرة. إذا كان عليّ الذهاب من 1 إلى 10 وأتقدم 1 في كل مرة، فالخطوة 1. إذا تقدمت 2، 4، 6، 8، 10، فالخطوة 2. أطلب من المستخدم ثلاث قيم: **من** **أين** أبدأ العد، و**أين** أنتهي، و**الخطوة**.

"حتى **ما** **قبل** النهاية" تعني أن عدد النهاية **لا** **يُطبع**: من 1 إلى 10 بخطوة 1 أطبع من 1 إلى 9. إذن الشرط في for هو `i < arrivo`، وليس `i <= arrivo`.

5.2 التصريح عن عدة متغيرات معًا

أحتاج إلى ثلاثة متغيرات. يمكن التصريح عنها بطريقتين، وهما **الشيء** **نفسه**: اختر ما يعجبك.

```
int partenza;
int arrivo;
int passo;

int partenza, arrivo, passo;
```

5.3 ما الذي يجب التحقق منه

في البداية والنهاية **لا** **يوجد** **تحقق**: أي عدد يصلح، فيكفي printf وscanf. أما **الخطوة** فيجب أن تكون **أكبر** **من** **0**: للتحقق منها نحتاج إلى **do** **while**، وdo while يتكرر عندما يكون العدد **خاطئًا**. الخاطئ هو **الأصغر** **من** **0** **أو** **المساوي** **له**:

```
do
{
    printf("Inserisci il passo maggiore di 0: ");
    scanf("%d", &passo);
} while (passo <= 0);
```

تنبيه

لماذا 0 خاطئ أيضًا. مع خطوة 0 لا تتغير `i` أبدًا: يبقى الشرط `i < arrivo` صحيحًا دائمًا و**لا** **تنتهي** الحلقة أبدًا. لهذا نكتب في التحقق `passo <= 0` وليس فقط `passo < 0`.

5.4 حلقة for مع الخطوة

يبدأ for من البداية `(int i = partenza)`، ويستمر ما دام `i < arrivo`، وفي كل دورة يضيف الخطوة: `i += passo`. لا يمكن استخدام `i++`، الذي يزيد دائمًا بمقدار 1. إذا كانت `i` تساوي 3 والخطوة 2، ففي الدورة التالية تصبح `i` تساوي 5، ثم 7، ثم 9، 11، 13، 15.

في printf نترك **مسافة** بعد %d، لتُقرأ الأعداد بشكل أفضل.

ملاحظة

تحقق من المتغيرات واحدًا واحدًا. عند نسخ سطور printf وscanf من السهل ترك الاسم الخاطئ: إذا كُتب `&arrivo` حيث يلزم `&partenza`، يعمل البرنامج لكن الحسابات لا تصح. إذا كانت الأعداد غريبة، أعد قراءة هذه السطور أولًا.

5.5 العدّاد

لا أريد رؤية الأعداد فقط: أريد أن أعرف **كم** **مرة** دارت الحلقة. نحتاج إلى متغير جديد، **العدّاد**، نسميه `cicli`. القواعد ثلاث:

- **قبل** الحلقة قيمته 0: `cicli = 0`؛
- **داخل** الحلقة، كلما طبعت شيئًا يزيد بمقدار 1: `++cicli`؛
- **بعد** الحلقة أطبعه مرة واحدة.

```c
#include <stdio.h>

int main()
{
    int partenza, arrivo, passo;
    int cicli;
    cicli = 0;

    printf("Inserisci il numero di partenza: ");
    scanf("%d", &partenza);

    printf("Inserisci il numero di arrivo: ");
    scanf("%d", &arrivo);

    do
    {
        printf("Inserisci il passo maggiore di 0: ");
        scanf("%d", &passo);
    } while (passo <= 0);

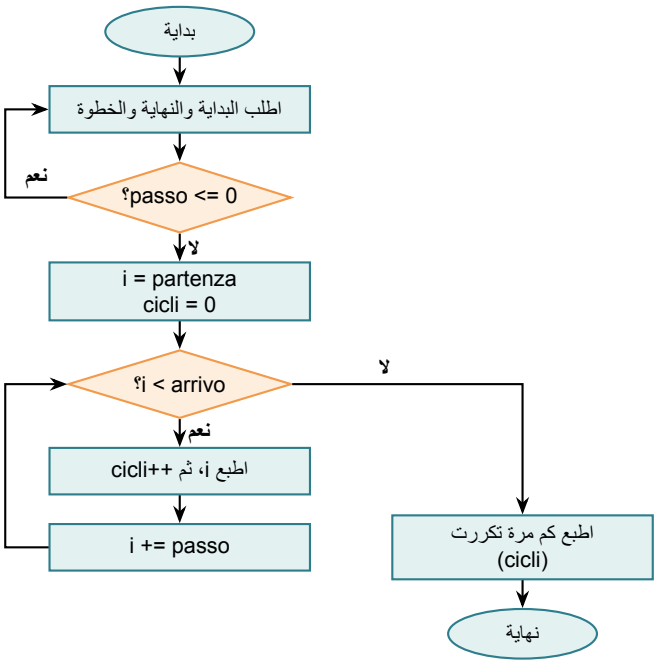
    for (int i = partenza; i < arrivo; i += passo)
    {
        printf("%d ", i);
        cicli++;
    }

    printf("\nAbbiamo ripetuto %d volte", cicli);

    return 0;
}
```

```
Inserisci il numero di partenza: 3
Inserisci il numero di arrivo: 17
Inserisci il passo maggiore di 0: 3
3 6 9 12 15
Abbiamo ripetuto 5 volte
```

جرّب أيضًا الحالات الغريبة. إذا كانت البداية أكبر من النهاية (مثلًا من 4 إلى 1 بخطوة 9)، يكون الشرط `i < arrivo` خاطئًا **فورًا**: لا تبدأ الحلقة، ولا يُطبع شيء، ويبقى العدّاد 0. هذا ليس خطأ: هذا بالضبط ما طلبناه.



شكل 4: مخطط انسيابي للعدّاد. يتحقق do while من الخطوة فقط: تُطلب البداية والنهاية قبله، بدون تحقق.

ملاحظة

التعليقات. يمكن كتابة أي شيء في الكود بعد الشرطتين `//`: هذا **تعليق**، وتتجاهله الآلة. يعمل البرنامج كما هو بدونه. التعليقات تفيد **من يقرأ**، لفهم ما يفعله البرنامج.

6 البرنامج الرابع: المضاعفات والمجموع

مثال

المطلوب. اطلب عددًا من 2 إلى 9 (تحقق من صحته). اطبع مضاعفات العدد حتى 100 وفي النهاية اطبع مجموع كل تلك الأعداد. مثلًا إذا أدخلت 7، سيطبع 7 14 21 ... 98 ثم المجموع، وهو 735.

هذا أيضًا جربتموه **أولًا بمفردكم**. do while هو نفسه دائمًا: يتكرر ما دام `(n < 2) || (n > 9)`.

6.1 المضاعفات

مع 7 أريد الأعداد 7، 14، 21، 28... في for:

- **يبدأ من العدد**: `int i = n;`

- **يستمر** **حتى** **100** مع 100: 100 => i؛
- في كل مرة **يضيف** **العدد**: n => i.

تنبيه

تزيد **i**، **وليس** **n**. كتابة i => n خطأ: n هو العدد الذي اختاره المستخدم ويجب أن **يبقى** **ثابتًا**. i هي التي تصبح 7، ثم 14، ثم 21. وتذكروا **الترتيب**: يُكتب do while **بعد** f وليس في أي مكان، لأننا نحتاج إلى العدد أولًا.

6.2 المجموع: المُراكِم

أريد مجموع كل الأعداد المطبوعة. أحتاج إلى متغير somma، قيمته **في** **البداية** **0**: إنها نقطة الانطلاق. كلما طبعت i، أضيفه:

```
somma += i;
```

وهذا هو نفسه somma = somma + i. مع 7 يحدث هذا:

i	7	14	21	28	35	...
somma بعد السطر	7	21	42	70	105	...

القيمة **تتراكم**. في نهاية الحلقة، **خارج** الأقواس المعقوصة، أطبع النتيجة.

```c
#include <stdio.h>

int main()
{
    int n;
    int somma;
    somma = 0;

    do
    {
        printf("Inserisci un numero da 2 a 9: ");
        scanf("%d", &n);
    } while ((n < 2) || (n > 9));

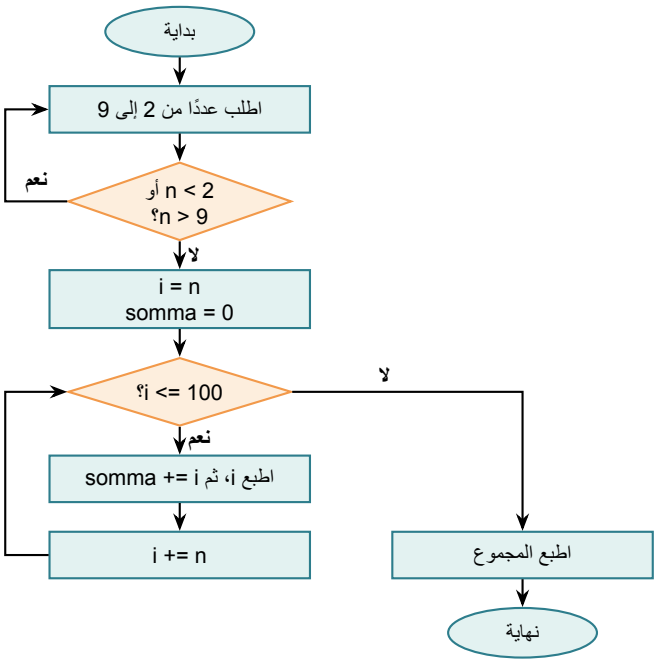
    for (int i = n; i <= 100; i += n)
    {
        printf("%d ", i);
        somma += i;
    }

    printf("\nLa somma di tutti i numeri vale %d", somma);

    return 0;
}
```

```
Inserisci un numero da 2 a 9: 7
7 14 21 28 35 42 49 56 63 70 77 84 91 98
La somma di tutti i numeri vale 735
```

جرّب دائمًا البرنامج بمثال المطلوب: إذا أدخلت 7 ولم تحصل على 735، فهناك ما يجب إصلاحه. إذا نسيت السطر `somma += i` مثلًا، تكون النتيجة 0.



شكل 5: مخطط انسيابي للمضاعفات مع المجموع.

6.3 مقارنة بين العدّاد والمجموع

المجموع (المُراكِم)	العدّاد	
كم تساوي القيم معًا	كم مرة دارت	ما فائدته
`somma = 0`	`cicli = 0`	قبل الحلقة
`somma += i` (القيمة الحالية)	`cicli++` (دائمًا 1 زيادة)	داخل الحلقة
يُطبع	يُطبع	بعد الحلقة

7 ملخص: حلقات for اليوم

ماذا يفعل	for	البرنامج
i تعد دائمًا من 1 إلى 10	`for (int i = 1; i <= 10; i++)`	جدول الضرب
ينزل من n إلى 1	`for (int i = n; i >= 1; i--)`	العد التنازلي
يقفز بمقدار passo، بدون النهاية	`for (int i = partenza; i < arrivo; i += passo)`	العدّاد
يقفز بمقدار n، مع 100	`for (int i = n; i <= 100; i += n)`	المضاعفات

قبل كتابة for، أجب عن ثلاثة أسئلة: **من أين أبدأ؟** ما دام ماذا **أستمر؟** كيف **أتغير** في كل دورة؟

8 كيف ندرس، وماذا يحدث الآن

في ثلاث ساعات أنجزنا **أربعة تمارين**: هذا قليل، فقد كانت خطة الصباح اثني عشر، ستة أقوم بها وستة تقومون بها. نحن نتعلم وهذا طبيعي، لكن هكذا **لن نتقدم**. أود الانتقال إلى أشياء أكثر تشويقًا وعملية، مثل **Arduino**، لكن لذلك يجب أن نكون **في المستوى نفسه** في كتابة البرامج. إذن:

- **الدراسة لا تعني** تصفّح الملف بسرعة. تعني قراءته **من البداية إلى النهاية**، والعودة إلى **التمارين التي أُنجزت في القسم**، و**محاولة إعادتها بمفردك**.
- **الواجبات المنزلية** تُسلَّم. بها أستطيع أن أفهم **أين المشكلة**: إذا بدأت جيدًا وأخطأت في for، أعرف أن علينا العمل على for. إذا كتب البرنامج ذكاء اصطناعي، لا أعرف ماذا أفعل لمساعدتك.

> **ملاحظة**
>
> **للمرة القادمة: الواجب C03**، مجموعة تمارين عن for والعدّاد والمجموع، تُسلَّم على Classroom. يسلّمها **الجميع**: ما تستطيع فعله، بالطريقة التي تستطيعها، دون أن تجعل الذكاء الاصطناعي **يكتب الكود**.
>
> **يوم الاثنين القادم** أود تجربة شيء: **تحدٍّ في أزواج**، مع **سائق** (**pilota**) يكتب و**ملّاح** (**navigatore**) يضع يديه في جيبيه ويقول له ماذا يفعل، كما في السيارة.

9 مسرد: كلمات إيطالية تُستخدم في القسم

- **tabellina**: جدول الضرب
- **Inserisci un numero da 1 a 10**: أدخل عددًا من 1 إلى 10
- **conto alla rovescia**: العد التنازلي
- **Via!**: انطلق!
- **partenza / arrivo / passo**: البداية / النهاية / الخطوة
- **contatore, cicli**: العدّاد، عدد الدورات
- **Abbiamo ripetuto ... volte**: كررنا ... مرات
- **multipli**: المضاعفات
- **somma**: المجموع
- **La somma di tutti i numeri vale**: مجموع كل الأعداد يساوي
- **decremento**: الإنقاص